Лекція №22 (2 години). Робота з базами даних (BaaS: Firebase / Supabase).

План лекції

1. Навіщо потрібні бази даних (БД) і які вони бувають (SQL vs NoSQL).
2. Проблема класичного бекенду та концепція BaaS (Backend-as-a-Service).
3. Огляд екосистеми Firebase (від Google).
4. Огляд платформи Supabase (Open-source альтернатива).
5. Налаштування проекту та підключення клієнта бази.
6. Основи CRUD операцій (Створення, Читання, Оновлення, Видалення).
7. Авторизація користувачів (Authentication) "з коробки".
8. RLS (Row Level Security) — політики безпеки даних.

Перелік умовних скорочень

- **DB/БД** (Database) — база даних.
- **SQL** (Structured Query Language) — реляційні бази даних із таблицями.
- **NoSQL** — нереляційні бази даних (документні, графі тощо).
- **BaaS** (Backend-as-a-Service) — бекенд як послуга (хмарна платформа).
- **CRUD** (Create, Read, Update, Delete) — базові операції з даними.
- **RLS** (Row Level Security) — безпека на рівні рядків.

Вступ

На минулій лекції ми навчилися створювати власний сервер на Node.js та Express. Ми зберігали масив користувачів (`const users = []`) прямо в пам'яті (в коді) сервера. Але що станеться, якщо наш комп'ютер перезавантажиться? Всі ці дані зникнуть назавжди.

Щоб дані жили надійно і зберігалися після вимкнення світла, їх потрібно складати у спеціальну програму — **Базу Даних (БД)**. Традиційно налаштування бази даних, написання бекенду для підключення до неї, створення безпечного зберігання паролів користувачів — це місяці роботи для команди Backend-інженерів.

Але що робити Frontend-розробнику, який хоче зробити власний повноцінний стартап за вихідні, не витрачаючи час на налаштування серверів Linux? На допомогу приходить концепція **BaaS (Бекенд як**

Послуга).

Мета цієї лекції — познайомитися з двома світовими лідерами на ринку BaaS (Firebase та Supabase) і навчитися зберігати дані в хмарі без написання власного сервера на Node.js!

1. Які бувають Бази Даних (SQL vs NoSQL)

Усі бази даних у світі глобально діляться на два табори:

1. Реляційні (SQL): PostgreSQL, MySQL Такі бази виглядають як ідеальні таблиці в Excel. У них жорстка структура (схема). Якщо ви створили таблицю "Користувачі" з колонками "Ім'я" та "Вік", ви не зможете просто так запхати туди третю характеристику (наприклад, "Улюблений колір") без зміни фізичної схеми таблиці. Усі таблиці пов'язані між собою строгими зв'язками. Треба знати спеціальну мову SQL.

2. Нереляційні (NoSQL): MongoDB, Firestore Зберігають дані не в таблицях, а у вигляді "Документів", які візуально дуже нагадують звичні нам об'єкти **JSON**. У них немає жорсткої схеми: один користувач може мати лише ім'я, а інший — ім'я і масив з 5 машинами. Дуже гнучкі і легкі на старті.

2. Концепція BaaS (Backend-as-a-Service)

BaaS — це хмарний сервіс, який каже вам: "*Ей, Frontend-розробнику! Не пиши свій Node.js/PHP сервер. Ми вже все написали! Ми даємо тобі готову Базу Даних в Інтернеті, готову систему Реєстрації (з Google/Facebook), готовий диск для зберігання картинок. Ось тобі наш JS-пакет, просто викликай наші функції прямо зі свого React-компонента!*"

Ви підключаєте SDK (бібліотеку) безпосередньо у Ваш браузерний React-код (або мобільний додаток) і спілкуєтеся з базою даних напряму. Це економить тисячі годин розробки для малого бізнесу і стартапів. Де факто це архітектура Serverless.

3. Огляд Firebase (Firestore)

Firebase — це найстаріший і найвідоміший BaaS, куплений компанією Google багато років тому. Його база даних називається `Firestore` — це **NoSQL** база. Вона працює за принципом Колекцій та Документів (Колекція "Users" містить мільйони Документів — конкретних користувачів).

Плюси:

- Легендарна надійність серверів Google.
- Дуже крута функція `Realtime`: якщо хтось залишить коментар, він миттєво з'явиться на екранах усіх інших користувачів без оновлення сторінки (через WebSockets "з коробки").
  Мінуси:

- Складно робити складні пошукові запити (наприклад: "знайди всі товари, де ціна > 100 І знижка < 5%"). NoSQL бази погано підходять для складної аналітики.
- "Vendor Lock-in" (Ви замкнені в екосистемі Google. Її код закритий).

4. Огляд Supabase

Supabase — це молодий, скажено популярний "вбивця Firebase". Його головна фішка в тому, що він **Open-Source** (відкрий код). А під капотом у нього працює класична, надпотужна, реляційна **SQL база даних — PostgreSQL**.

Плюси:

- Використовує Postgres (Золотий стандарт баз даних світового рівня). Ви можете будувати неймовірно складні зв'язки між даними.
- Ви не замкнені (No Vendor Lock-in). Ви можете будь-якої миті завантажити всю базу і розгорнути її на власному комп'ютері.
- Чудова документація з готовим JS-кодом для React.
- Надає власним API-пакет `supabase-js`, який звертається до бази так само легко, як і Firebase.

> *На практиці ми будемо використовувати саме **Supabase**, як кращий сучасний інженерний стандарт.*

5. Налаштування проекту (Supabase)

1. Зайти на `supabase.com`, створити новий Project (безкоштовно).
2. Зайти в налаштування проекту і знайти дві константи `Project URL` та `anon public key` (Анонімний API-ключ).
3. У своєму React-проекті (через термінал) встановити їхню JS бібліотеку-клієнт: `npm install @supabase/supabase-js`
4. Створити у вашому файлі `src/supabaseClient.js` підключення:

```
import { createClient } from "@supabase/supabase-js";

// Ніколи не пишіть ключі прямо в коді! Зберігайте їх у файлі `.env` (прихованому від Git)
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;
```

```
// Створюємо головний об'єкт для роботи з базою
export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

6. Основи CRUD операцій

Всі маніпуляції з даними в `supabase-js` асинхронні (через `await`) і дуже нагадують SQL, але пишуться мовою JavaScript (JS Builder).

Уявімо, що ви створили в інтерфейсі Supabase таблицю `posts` (пости в блозі).

Читання (Read - GET)

```
// Знайти всі пости (select '*') і завантажити їх масивом у змінну data
const { data, error } = await supabase.from("posts").select("*");

// Знайти лише пости від певного автора, відсортовані за датою
const { data, error } = await supabase
  .from("posts")
  .select("title, content") // Просимо не всі колонки, а тільки ці 2
  .eq("author_id", 123) // eq = Equal (дорівнює)
  .order("created_at", { ascending: false }); // Старіші знизу
```

Створення (Create - POST)

```
const newPost = { title: "Привіт світ!", author_id: 123 };

const { data, error } = await supabase.from("posts").insert([newPost]).select();
// Якщо хочемо, щоб база повернула назад готовий запис з новим ID
```

Оновлення та Видалення (Update / Delete)

```
// Оновити статтю з ідентифікатором 5
const { error } = await supabase
  .from("posts")
  .update({ title: "Виправлений заголовок" })
  .eq("id", 5);
```

```
// Видалити статтю
const { error } = await supabase.from("posts").delete().eq("id", 5);
```

Вам взагалі не потрібно було писати сервер на Node.js! Ви викликаєте цей код прямо з клієнтської кнопки `onClick={handleDelete}` у React (звісно, огорнувши його в `try..catch`).

7. Авторизація користувачів (Authentication)

Створення надійної системи логіну (з хешуванням паролів, відправкою листів для відновлення, підтвердженням email) — це місяці страждань. З BaaS-платформами це робиться одним рядком коду, бо сервери Supabase роблять все під капотом!

Реєстрація:

```
const { data, error } = await supabase.auth.signUp({
  email: "example@email.com",
  password: "example-password",
});
```

Якщо виклик успішний, Supabase створить юзера в базі даних, автоматично захешує пароль і (якщо налаштовано) надішле на цю пошту лист із посиланням `Confirm Email`.

Логін:

```
const { data, error } = await supabase.auth.signInWithPassword({
  email: "example@email.com",
  password: "example-password",
});
```

Після логіну Supabase автоматично збереже зашифрований JWT-токен сесії у вашому браузері (Local Storage). Тепер ваш браузер "авторизовано" надійно.

Перевірка поточного юзера:

```
// Щоб дізнатися (наприклад у компоненті Хедера), чи увійшов хтось на сайт
const {
  data: { user },
} = await supabase.auth.getUser();

if (user) {
```

```
  console.log("Привіт, твій ID:", user.id);
}
```

8. Захист даних: RLS (Row Level Security)

Виникає очевидне і дуже розумне питання: *"Якщо ми даємо можливість JavaScript-коду (який лежить відкритим у браузері) напряму підключатися і змінювати базу даних, чи може розумний хакер відкрити Консоль (F12) і написати команду* `supabase.from('posts').delete()` *, і видалити базу магазину?"*

ТАК! Зможе! Більше того, за замовчуванням в Supabase доступ заблоковано для всіх. Щоб дозволити чесні звернення, ви повинні налаштувати **RLS (Row Level Security)**. RLS — це своєрідні "Правила поведінки" (Policies), які ви пишете безпосередньо в панелі управління БД на сайті Supabase.

Наприклад ви кажете:

- "Дозволено ЧИТАТИ (Select) таблицю `posts` ВСІМ АНОНІМНИМ людям".
- "Але дозволено ДОДАВАТИ (Insert) нові рядки ТІЛЬКИ ТИМ людям, у яких є авторизаційний токен (`auth.uid()`).
- "А дозволено ВИДАЛЯТИ або РЕДАГУВАТИ статтю тільки тому юзеру, чий `ID` співпадає з `author_id` конкретної статті!".

Тепер, якщо хакер спробує зробити `delete()` , сервер Supabase перевірить його токен з браузера. Якщо хакер є автором статті — видалить. Якщо не він робив цю статтю — сервер дасть по рукам (Помилка "403 Forbidden Access"), не довіряючи надісланій з JS-клієнта команді.

Саме RLS дозволяє будувати безпечні Serverless додатки без свого бекенд-сервера.

Висновки

1. Для постійного (персистентного) збереження інформації веб-додатки використовують Бази Даних. Ринок поділяється на суворі табличні (SQL) та гнучкі документальні (NoSQL).
2. Історично для з'єднання з базою потрібен був сервер (Backend). Парадигма **BaaS (Backend-as-a-Service)** усунула цю ланку, дозволивши розробникам Frontend'у робити маніпуляції та реєстрацію юзерів напряму з клієнта.
3. Firebase — це найстаріша платформа від Google на основі NoSQL. Вона чудова своєю "Realtime" синхронізацією з коробки, але має проблеми з фільтрацією даних та закритим вихідним кодом.
4. **Supabase** стрімко набув статусу світового стандарту завдяки тому, що є відкритою (Open-source) оболонкою над надійною SQL-базою (PostgreSQL).
5. Будь-які CRUD-операції в Supabase зводяться до виклику асинхронних JavaScript-функцій клієнта `.select()` , `.insert()` , `.delete()` всередині ваших React-компонентів.

6. Модуль Аутентифікації (`supabase.auth`) безкоштовно та надійно вирішує біль із хешуванням паролів, відправкою листів і зберіганням сесій (JWT-токенів) у LocalStorage користувача.
7. Відкритість бази для браузерного JS-клієнта зобов'язує розробників прописувати правила доступу **Row Level Security (RLS)** безпосередньо на серверах БД для запобігання несанкціонованим маніпуляціям даними.

Джерела

1. Supabase JS Client Docs — як читати/редагувати/видаляти дані.
2. What is Backend as a Service? (BaaS)
3. Row Level Security in Supabase — навіщо потрібен RLS і як він запобігає хаку бази.

Запитання для самоперевірки

1. Чим фундаментально відрізняється таблиця SQL (наприклад PostgreSQL) від колекції NoSQL (наприклад Firestore)? Яка з цих архітектур менш гнучка до зміни кількості характеристик сутності на льоту?
2. Що таке BaaS? Як цей підхід рятує життя Frontend-інженерам чи мобільним розробникам-одинакам під час створення власних проектів та стартапів?
3. В чому головна архітектурна різниця "під капотом" між конкурентами Firebase та Supabase? Яка база є реляційною?
4. Які змінні конфігурації (які зазвичай отримують в адмін-панелі) обов'язково треба передати функції `createClient()` для коннекту клієнта з хмарою, і де прийнято їх приховувати в проекті?
5. Напишіть за допомогою чейнінгу методів `supabase` , як зібрати і відсортувати список з 10 товарів.
6. Як працює Аутентифікація в Supabase: які дві базові функції викликаються для реєстрації юзера та для його логіну? Куди платформа непомітно зберігає "перепустку" (токен) в межах браузера?
7. Сценарій: Хакери можуть прочитати ваші ключі Supabase у відкомпільованому `bundle.js` сайті і спробувати викликати з консолі метод `supabase.delete()` , щоб обнулити вам базу. Який механізм інженерного захисту створений для блокування цього?